



Rationale Generation for Performance-Improving Code Changes

AV&V Course - A.Y. 25-26

February 23, 2026

Motivation

Automated generation of code changes rationale

- **Context:** automatically extracting the rationale from code changes is useful for many SE tasks, such as documentation generation, prompt augmentation in coding assistance, test case generation ...
- **Opportunity:** *Large Language Models* (LLMs) can automatically review code changes. However, they're better at explaining *what* changed, rather than *why*¹.
 - This is even harder when dealing with performance-related changes, that depend on an in-depth understanding of code semantics².

¹ F. Casillo, A. Mastropaolo, G. Bavota, V. Deufemia and C. Gravino, "Towards Generating the Rationale for Code Changes," ICPC. 2025 doi: 10.1109/ICPC66645.2025.00041.

² A. Shypula, et al. "Learning Performance-Improving Code Edits," ICLR 2024. <https://openreview.net/forum?id=ix7rLVHXyY>

Dataset

SWE-Perf (performance-improving code changes)

Real-world dataset composed of:

- **140** *curated* instances (performance-improving diffs)
- Repository, commit SHAs and messages
- Test classes and performance metrics

SWE-Perf: Can Language Models Optimize Code Performance on Real-World Repositories?

Xinyi He¹, Qian Liu^{2†}, Mingzhe Du³, Lin Yan², Zhijie Fan², Yiming Huang⁴
Zejian Yuan¹, Zejun Ma²

¹Xi'an Jiaotong University, ²TikTok

³National University of Singapore, ⁴University of California San Diego

Code performance optimization is paramount in real-world software engineering and critical for production-level systems. While Large Language Models (LLMs) have demonstrated impressive capabilities in code generation and bug fixing, their proficiency in enhancing code performance at the repository level remains largely unexplored. To address this gap, we introduce SWE-Perf, the first benchmark specifically designed to systematically evaluate LLMs on code performance optimization tasks within authentic repository contexts. SWE-Perf comprises 140 carefully curated instances, each derived from performance-improving pull requests from popular GitHub repositories. Each benchmark instance includes the relevant codebase, target functions, performance-related tests, expert-authored patches, and executable environments. Through a comprehensive evaluation of representative methods that span file-level and repo-level approaches (e.g., Agentless and OpenHands), we reveal a substantial capability gap between existing LLMs and expert-level optimization performance, highlighting critical research opportunities in this emerging field.

 **Date:** July 16, 2025

 **Home:** <https://swe-perf.github.io>

 **Correspondence:** Qian Liu (qian.liu@tiktok.com)



Simple code change example

Early None check

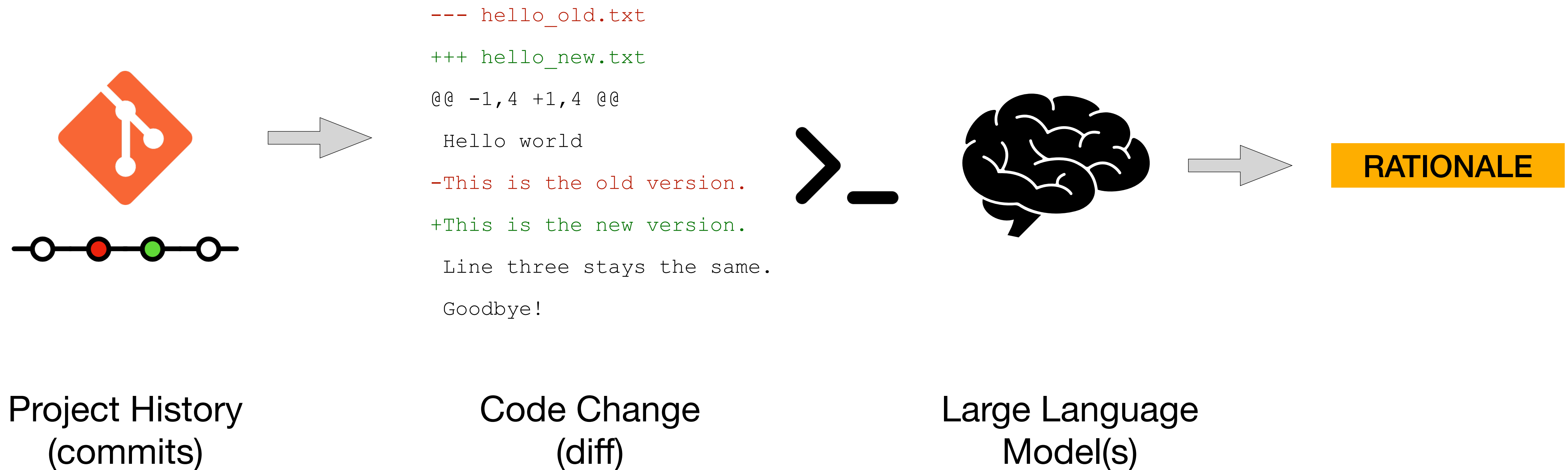
str(units) is
evaluated once.

```
def _is_time_like(units):
    # test for time-like
+   if units is None:
+       return False
    time_strings = [
-       "since",
        "days",
        "hours",
        "minutes",
@@ -227,7 +228,19 @@ def _is_time_like(units):
        "microseconds",
        "nanoseconds",
    ]
-   return any(tstr in str(units) for tstr in time_strings)
+   units = str(units)
+   # to prevent detecting units like `days accumulated` as time-like
+   # special casing for datetime-units and timedelta-units (GH-8269)
+   if "since" in units:
+       from xarray.coding.times import _unpack_netcdf_time_units
+
+       try:
+           _unpack_netcdf_time_units(units)
+       except ValueError:
+           return False
+       return True
+   else:
+       return any(tstr == units for tstr in time_strings)

class CFMaskCoder(VariableCoder):
```

Step 1 - Rationale Generation with LLMs

Extracting the intent and potential risk of the change



Example Prompt

Ask the model to provide the change rationale

Context

*You are an AI software engineer.
Below is a GitHub diff that claims to improve performance.*

Diff

`< code change />`

Questions

What problem does it solve? (Intent)

Why was this particular optimization chosen? (Motivation)

Are the code functionally equivalent? Are there side effects? (Risk assessment)

Which type of code change is this? (Algo, data structure, synchronization)

Guidelines

Reply with format including the following fields ...

Be clear and concise. Do not include

Step 2 - Evaluation

Inspection of the results

- What types of performance-improving changes are included in this dataset?
Is it possible to group them in few categories?
- How much is the explanation accurate?
 - Is it correct?
 - Is it comprehensive?
 - Is it coherent wrt the commit message?

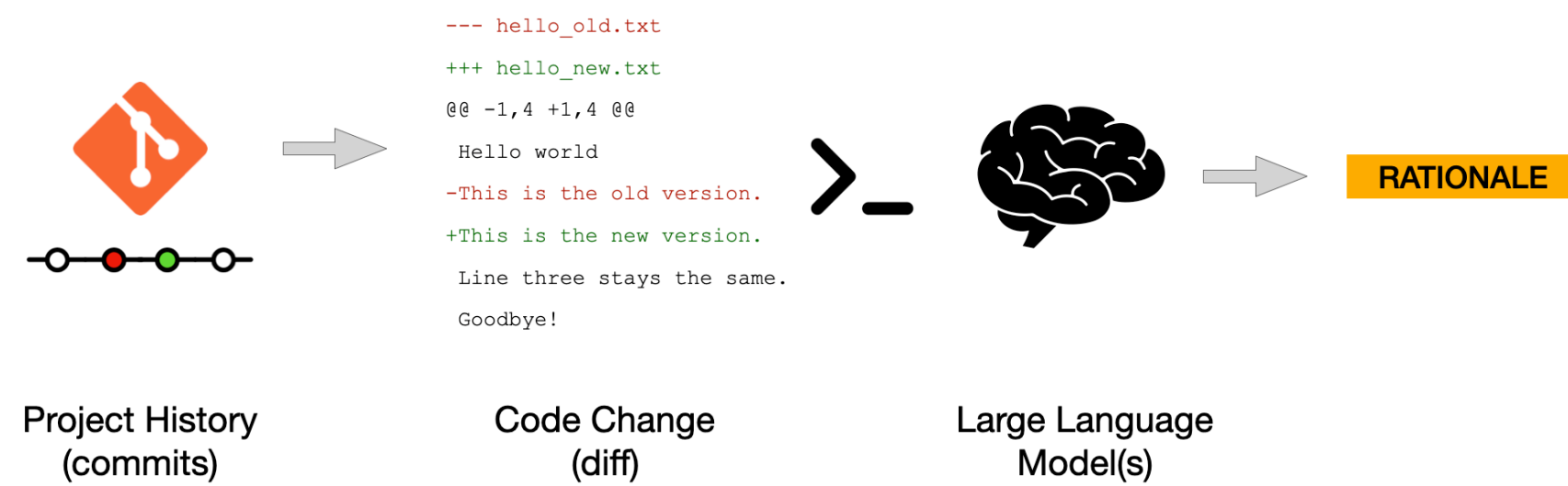
Requirements

... in addition to software quality and performance concepts

- Technical knowledge on python language and popular libs for data analysis and ML (numpy, pytorch...)
- GitHub workflows: commit, issues, pull requests
- General understanding on ML pipelines and Language Models (e.g., tokenization, pre-training and fine-tuning)

Step 1 - Rationale Generation with LLMs

Extracting the intent and potential risk of the change



Step 2 - Evaluation

Inspection of the results

- What types of performance-improving changes are included in this dataset?
Is it possible to group them in few categories?
- How much is the explanation accurate?
 - Is it correct?
 - Is it comprehensive?
 - Is it coherent wrt the commit message?

Questions?